

Database Systems		
Lab #:	09	
Lab Title:	Implementation of DML commands of SQL with suitable examples	
Submitted by:		
Names		Registration #
Sumaira Safeer		FA22-BCE-019

Rubrics name & number					Marks	
					In-Lab	Post-Lab
Engineering Knowledge	R2: Use of Engineering Knowledge and follow Experiment Procedures: Ability to follow experimental procedures, control variables, and record procedural steps on lab report.					
Problem Analysis	R6: Experimental Data Analysis: Ability to interpret findings, compare them to values in the literature, identify weaknesses and limitations.					
Design	R8: Best Coding Standards: Ability to follow the coding standards and programming practices.					
Modern Tools Usage	R9: Understand Tools: Ability to describe and explain the principles behind and applicability of engineering tools.					
Individual and Teamwork	R12: Individual Work Contributions: Ability to carry out individual responsibilities.					
	R13: Management of Team Work: Ability to appreciate, understand and work with multidisciplinary team members.					
Rubrics #	R2	R6	R8	R9	R12	R13
In Lab						
Post- Lab						

Lab No: 09

Title: Implementation of DML commands of SQL with suitable examples.

- Insert table.
- Update table.
- Delete Table.

Implementation of different types of functions with suitable examples:

- Number Function.
- Aggregate Function.

Objective:

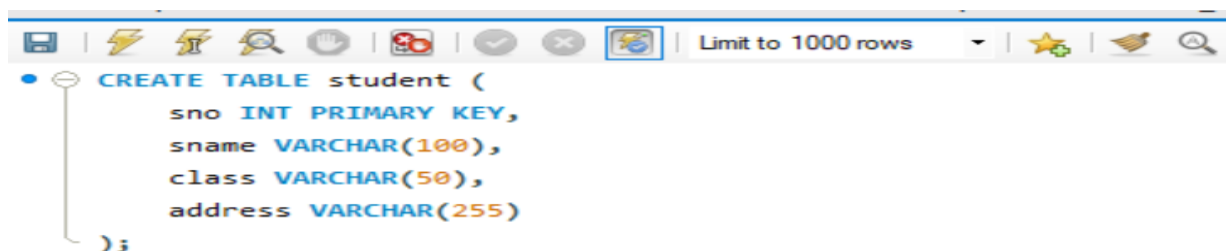
To understand the different issues involved in the design and implementation of a database system.

To understand and use data manipulation language to query, update, and manage a database.

To understand and implement various types of function in SQL.

InLab Tasks:

Inserting the Student Records:



```

7 • select * from student;
8 • INSERT INTO student (sno, sname, class, address) VALUES
9   (1, 'Sumaira Safeer', '10th Grade', 'Karachi, Sindh, Pakistan'),
10  (2, 'Rohan Khan', '12th Grade', 'Lahore, Punjab, Pakistan'),
11  (3, 'Ali Mustafa', '9th Grade', 'Islamabad, Capital Territory, Pakistan'),
12  (4, 'Nayab Bibi', '10th Grade', 'Rawalpindi, Punjab, Pakistan'),
13  (5, 'Kamran Hassan', '11th Grade', 'Multan, Punjab, Pakistan'),
14  (6, 'Yasir Malik', '12th Grade', 'Faisalabad, Punjab, Pakistan'),
15  (7, 'Maria Farooq', '8th Grade', 'Peshawar, Khyber Pakhtunkhwa, Pakistan'),
16  (8, 'Mariam Nasir', '9th Grade', 'Quetta, Balochistan, Pakistan'),
17  (9, 'Hashim Ahmed', '11th Grade', 'Karachi, Sindh, Pakistan'),
18  (10, 'Amna Iqbal', '10th Grade', 'Sialkot, Punjab, Pakistan');

```

✓	1	14:16:25	CREATE TABLE student (sno INT PRIMARY KEY, sname VARCHAR(100)...	0 row(s) affected	0.031 sec
✓	2	14:16:25	select * from student LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
✓	3	14:16:25	INSERT INTO student (sno, sname, class, address) VALUES (1, 'Ahmed Ali', '10t...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.016 sec

Inserting the product Records:

```

20 • CREATE TABLE product (
21     products_id INT PRIMARY KEY,
22     product_name VARCHAR(50),
23     pro_code INT,
24     price DECIMAL(10, 2)
25 );

26 • INSERT INTO product (products_id, product_name, pro_code, price) VALUES
27   (1, 'T_shirt', 768, 7800),
28   (2, 'shirt', 124, 599),
29   (3, 'lipstick', 948, 800),
30   (4, 'lense', 7976, 1500),
31   (5, 'jeans', 234, 1200),
32   (6, 'jacket', 451, 3000),
33   (7, 'shoes', 867, 4500),
34   (8, 'hat', 342, 750),
35   (9, 'belt', 556, 999),
36   (10, 'sunglasses', 221, 2000);

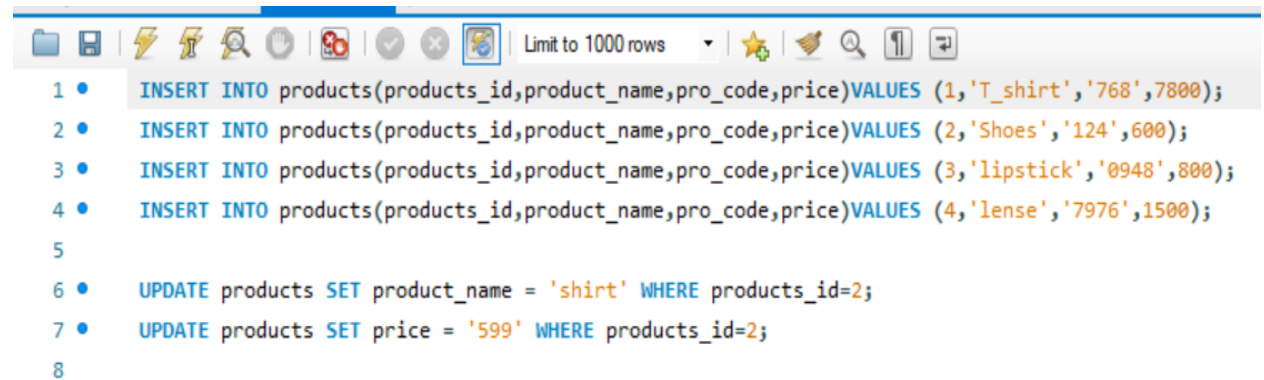
```

Task1:

- Update the Shirt with shoes by applying condition on Pro_code in product detail table
- Update the T_Shirt with Shirt and 599 with 600 by applying condition on Pro_code.

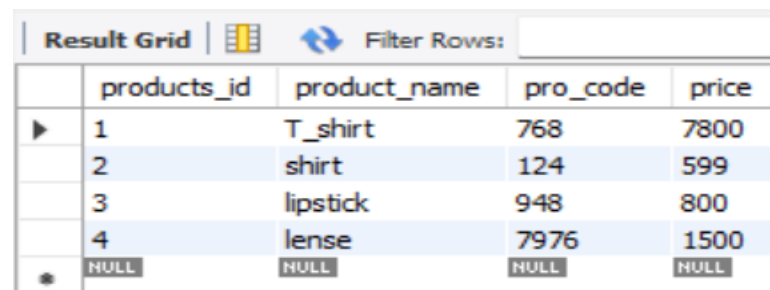
Syntax: SQL>UPDATE relation name SET Field_name1=data,field_name2=data,

WHERE field_name=data;



```
1 • INSERT INTO products(products_id,product_name,pro_code,price)VALUES (1,'T_shirt','768',7800);
2 • INSERT INTO products(products_id,product_name,pro_code,price)VALUES (2,'Shoes','124',600);
3 • INSERT INTO products(products_id,product_name,pro_code,price)VALUES (3,'lipstick','0948',800);
4 • INSERT INTO products(products_id,product_name,pro_code,price)VALUES (4,'lense','7976',1500);
5
6 • UPDATE products SET product_name = 'shirt' WHERE products_id=2;
7 • UPDATE products SET price = '599' WHERE products_id=2;
8
```

Result Grid:



	products_id	product_name	pro_code	price
▶	1	T_shirt	768	7800
	2	shirt	124	599
	3	lipstick	948	800
	4	lense	7976	1500
•	NULL	NULL	NULL	NULL

Task 2:

- Delete all records from Product-detail.

Deleting a record:

```

39
40 • select * from product;
41 • DELETE FROM product;
42

```

Result Grid				Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
products_id	product_name	pro_code	price				
NULL	NULL	NULL	NULL				

✓	13	22:25:11	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
✓	14	22:25:11	DELETE FROM product	10 row(s) affected	0.015 sec
✓	15	22:25:11	SET SQL_SAFE_UPDATES = 1	0 row(s) affected	0.000 sec
✓	16	22:25:17	select * from product LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec

- Delete a record from Student_info where St_id=1

Deleting a record:

```

38
39 • DELETE FROM student WHERE sno = 1;
40

```

Result Grid				Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
sno	sname	class	address				
2	Sara Khan	12th Grade	Lahore, Punjab, Pakistan				
3	Ali Shah	9th Grade	Islamabad, Capital Territory, Pakistan				
4	Fatima Bibi	10th Grade	Rawalpindi, Punjab, Pakistan				
5	Hassan Raza	11th Grade	Multan, Punjab, Pakistan				
6	Zainab Malik	12th Grade	Faisalabad, Punjab, Pakistan				
7	Omar Farooq	8th Grade	Peshawar, Khyber Pakhtunkhwa, Pakistan				
8	Mariam Nasir	9th Grade	Quetta, Balochistan, Pakistan				
9	Bilal Ahmed	11th Grade	Karachi, Sindh, Pakistan				

#	Time	Action	Message	Duration / Fetch
✓	13	22:25:11	SET SQL_SAFE_UPDATES = 0	0 row(s) affected
✓	14	22:25:11	DELETE FROM product	10 row(s) affected
✓	15	22:25:11	SET SQL_SAFE_UPDATES = 1	0 row(s) affected
✓	16	22:25:17	select * from product LIMIT 0, 1000	0 row(s) returned
✓	17	22:31:29	INSERT INTO product (products_id, product_name, pro_code, price) VALUES (1, 'T_shirt', ...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
✓	18	22:31:34	select * from product LIMIT 0, 1000	10 row(s) returned

Task 3:

- Display all records of Product_detail and Student_info.

Syntax: SELECT * FROM relation_name;

Display of Student_info:

```
7 • select * from student;
8 • INSERT INTO student (sno, sname, class, address) VALUES
9   (1, 'Sumaira Safeer', '10th Grade', 'Karachi, Sindh, Pakistan'),
10  (2, 'Rohan Khan', '12th Grade', 'Lahore, Punjab, Pakistan'),
11  (3, 'Ali Mustafa', '9th Grade', 'Islamabad, Capital Territory, Pakistan'),
12  (4, 'Nayab Bibi', '10th Grade', 'Rawalpindi, Punjab, Pakistan'),
13  (5, 'Kamran Hassan', '11th Grade', 'Multan, Punjab, Pakistan'),
14  (6, 'Yasir Malik', '12th Grade', 'Faisalabad, Punjab, Pakistan'),
15  (7, 'Maria Farooq', '8th Grade', 'Peshawar, Khyber Pakhtunkhwa, Pakistan'),
16  (8, 'Mariam Nasir', '9th Grade', 'Quetta, Balochistan, Pakistan'),
17  (9, 'Hashim Ahmed', '11th Grade', 'Karachi, Sindh, Pakistan'),
18  (10, 'Amna Iqbal', '10th Grade', 'Sialkot, Punjab, Pakistan');
```

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
sno	sname	class	address	
1	Ahmed Ali	10th Grade	Karachi, Sindh, Pakistan	
2	Sara Khan	12th Grade	Lahore, Punjab, Pakistan	
3	Ali Shah	9th Grade	Islamabad, Capital Territory, Pakistan	
4	Fatima Bibi	10th Grade	Rawalpindi, Punjab, Pakistan	
5	Hassan Raza	11th Grade	Multan, Punjab, Pakistan	
6	Zainab Malik	12th Grade	Faisalabad, Punjab, Pakistan	
7	Omar Farooq	8th Grade	Peshawar, Khyber Pakhtunkhwa, Pakistan	
8	Mariam Nasir	9th Grade	Quetta, Balochistan, Pakistan	

Display of Product_detail:

Limit to 1000 rows

```

35 (6, 'jacket', 451, 3000),
36 (7, 'shoes', 867, 4500),
37 (8, 'hat', 342, 750),
38 (9, 'belt', 556, 999),
39 (10, 'sunglasses', 221, 2000);
40 • select * from product;

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap

	products_id	product_name	pro_code	price
▶	1	T_shirt	768	7800.00
	2	shirt	124	599.00
	3	lipstick	948	800.00
	4	lense	7976	1500.00
	5	jeans	234	1200.00
	6	jacket	451	3000.00
	7	shoes	867	4500.00
	8	hat	342	750.00

✓	15	22:25:11	SET SQL_SAFE_UPDATES = 1	0 row(s) affected	0.000 sec
✓	16	22:25:17	select * from product LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
✓	17	22:31:29	INSERT INTO product (products_id, product_name, pro_code, price) VALUES (1, 'T_shirt', ...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.000 sec
✓	18	22:31:34	select * from product LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec

Task 4:

- Display St_name and address form Student_info.

Syntax: SELECT a set of fields FROM relation_name;

43

44 • `select sname, address from student;`

45

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	sname	address
▶	Sara Khan	Lahore, Punjab, Pakistan
	Ali Shah	Islamabad, Capital Territory, Pakistan
	Fatima Bibi	Rawalpindi, Punjab, Pakistan
	Hassan Raza	Multan, Punjab, Pakistan
	Zainab Malik	Faisalabad, Punjab, Pakistan
	Omar Farooq	Peshawar, Khyber Pakhtunkhwa, Pakistan
	Mariam Nasir	Quetta, Balochistan, Pakistan
	Bilal Ahmed	Karachi, Sindh, Pakistan

student 10 x

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
13	22:25:11	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec

Task 5:

- Display the record of each Student whose class is 9th Grade.

Syntax: SELECT a set of fields FROM relation_name WHERE condition;

```

44
45 • SELECT sno, sname, class, address FROM student WHERE class = '9th Grade';
46

```

Result Grid				
sno	sname	class	address	
3	Ali Shah	9th Grade	Islamabad, Capital Territory, Pakistan	
8	Mariam Nasir	9th Grade	Quetta, Balochistan, Pakistan	
NULL	NULL	NULL	NULL	

Action Output				
#	Time	Action	Message	Duration / Fetch
21	22:40:40	select * from student LIMIT 0, 1000	9 row(s) returned	0.000 sec / 0.000 sec
22	22:41:48	select sname, address from student LIMIT 0, 1000	9 row(s) returned	0.000 sec / 0.000 sec
23	22:42:46	select * from student LIMIT 0, 1000	9 row(s) returned	0.000 sec / 0.000 sec
24	22:45:40	SELECT sname FROM student WHERE class = '9th Grade' LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
25	22:45:59	SELECT sname, class FROM student WHERE class = '9th Grade' LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
26	22:49:00	SELECT sno, sname, class, address FROM student WHERE class = '9th Grade' LIMIT 0, 10...	2 row(s) returned	0.016 sec / 0.000 sec

Task 6:

- Count the number of students in Student_info.

Count: COUNT following by a column name returns the count of tuple in that column.

Syntax: COUNT (Column name).

Example: SELECT COUNT (Sal) FROM emp;

```

46
47 • SELECT COUNT(*) FROM student;
48

```


Result Grid				Filter Rows:	Export:	Wrap Cell Content:
	COUNT(*)					
	9					

27	22:53:25	select COUNT (sname)from student LIMIT 0, 1000	Error Code: 1630. FUNCTION sys.COUNT does not exist. Check the 'Function Name Parsing...	0.015 sec
28	22:54:09	select COUNT (sno)from student LIMIT 0, 1000	Error Code: 1630. FUNCTION sys.COUNT does not exist. Check the 'Function Name Parsing...	0.000 sec
29	22:55:02	SELECT COUNT(*) FROM student LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec

Task 7:

- Add all the product price form the Product detail table.

SUM: SUM followed by a column name returns the sum of all the values in that column.

Syntax: SUM (Column name).

Example: SELECT SUM (Sal) From emp;

47

48 • `SELECT SUM(price) AS total_price FROM product;`

49

Result Grid				Filter Rows:	Export:	Wrap Cell Content:
	total_price					
	23148.00					

29	22:55:02	SELECT COUNT(*) FROM student LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
30	22:59:07	SELECT SUM(price) AS total_price FROM product LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

Task 8:

- Find the average age from the price_info table.

AVG: AVG followed by a column name returns the average value of that column values.

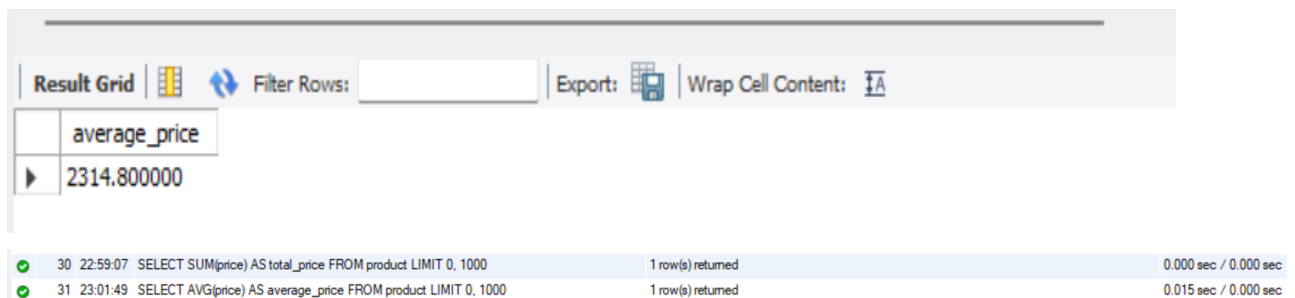
Syntax: AVG (n1, n2...)

Example: Select AVG (10, 15, 30) FROM DUAL;

48

49 • `SELECT AVG(price) AS average_price FROM product;`

50



The screenshot shows a database query result grid. The top toolbar includes 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. The result grid displays a single row with the column 'average_price' and the value '2314.800000'. Below the grid, a log shows two queries: a SUM query (row 30) and the current AVG query (row 31).

	average_price
▶	2314.800000

Icon	ID	Time	Query	Rows	Time
✓	30	22:59:07	SELECT SUM(price) AS total_price FROM product LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
✓	31	23:01:49	SELECT AVG(price) AS average_price FROM product LIMIT 0, 1000	1 row(s) returned	0.015 sec / 0.000 sec

Task 9:

- Find the maximum Pro_Price in the Product_detail table.

MAX: MAX followed by a column name returns the maximum value of that column.

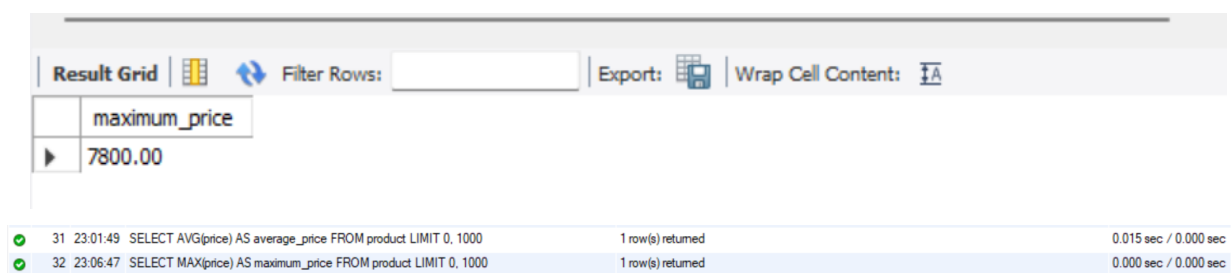
Syntax: MAX (Column name).

Example: SELECT MAX (Sal) FROM emp;

49

50 • `SELECT MAX(price) AS maximum_price FROM product;`

51



The screenshot shows a database query result grid. The top toolbar includes 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. The result grid displays a single row with the column 'maximum_price' and the value '7800.00'. Below the grid, a log shows two queries: an AVG query (row 31) and the current MAX query (row 32).

	maximum_price
▶	7800.00

Icon	ID	Time	Query	Rows	Time
✓	31	23:01:49	SELECT AVG(price) AS average_price FROM product LIMIT 0, 1000	1 row(s) returned	0.015 sec / 0.000 sec
✓	32	23:06:47	SELECT MAX(price) AS maximum_price FROM product LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

Task 10:

- Find the minimum Pro_Price in the Product_detail table.

MIN: MIN followed by column name returns the minimum value of that column.

Syntax: MIN (Column name).

Example: SELECT MIN (Sal) FROM emp;

51

52 • SELECT MIN(price) AS minimum_price FROM product;

53 -----

54

Result Grid			
Filter Rows:			
Export:  Wrap Cell Content: 			
	minimum_price		
	599.00		

32	23:06:47	SELECT MAX(price) AS maximum_price FROM product LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
33	23:08:12	SELECT MIN(price) AS minimum_price FROM product LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

Post lab task:

Task 1:

Write a difference between delete and truncate command.

Answer:

TRUNCATE:

- TRUNCATE is a DDL command
- TRUNCATE is executed using a table lock and whole table is locked for remove all records.
- We cannot use Where clause with TRUNCATE.
- TRUNCATE removes all rows from a table.
- Minimal logging in transaction log, so it is performance wise faster.
- TRUNCATE TABLE removes the data by deallocating the data pages used to store the table data and records only the page deallocations in the transaction log.
- Identity column is reset to its seed value if table contains any identity column.
- To use Truncate on a table you need at least ALTER permission on the table.
- Truncate uses the less transaction space than Delete statement.
- Truncate cannot be used with indexed views.

DELETE:

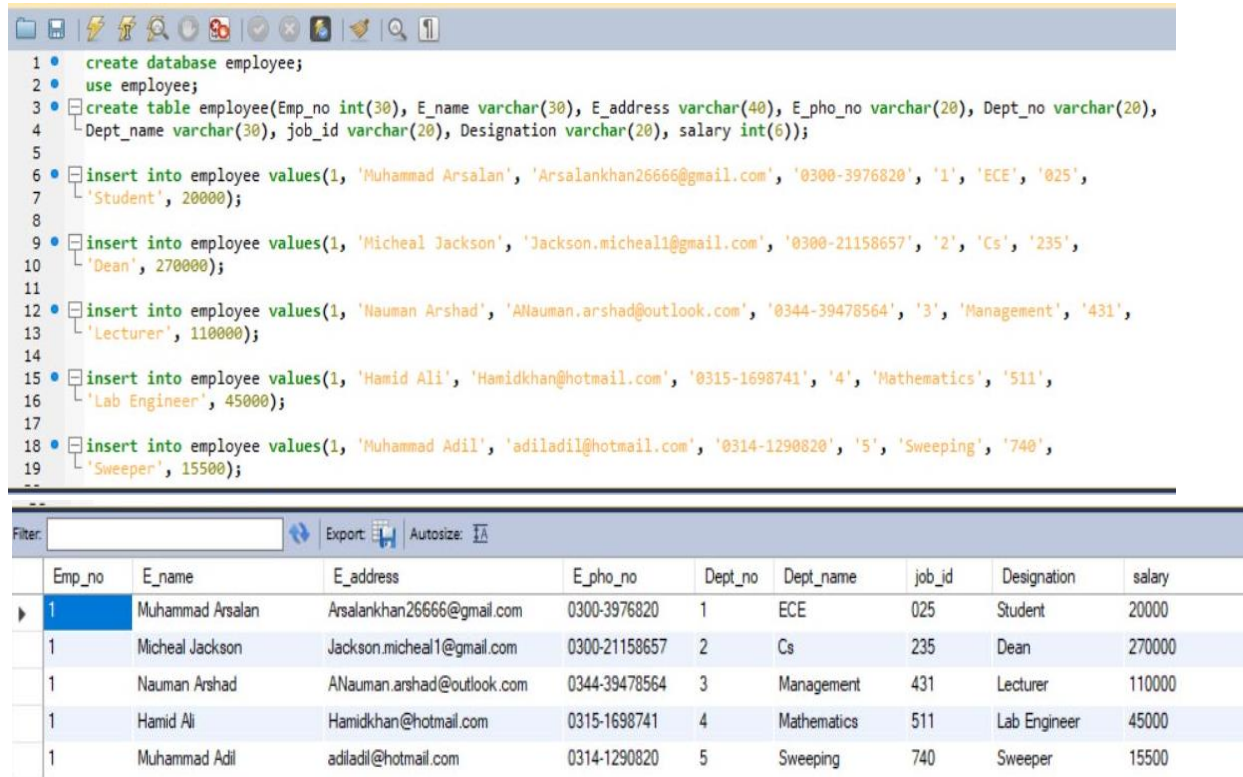
- DELETE is a DML command.
- DELETE is executed using a row lock, each row in the table is locked for deletion.
- We can use where clause with DELETE to filter & delete specific records.
- The DELETE command is used to remove rows from a table based on WHERE condition.
- It maintain the log, so it slower than TRUNCATE.
- The DELETE statement removes rows one at a time and records an entry in the transaction log for each deleted
- Identity of column keep DELETE retain the identity.
- To use Delete you need DELETE permission on the table.
- Delete uses the more transaction space than Truncate statement.
- Delete can be used with indexed views.

Task2:

Create a table EMPLOYEE with following schema:

(Emp_no, E_name, E_address, E_ph_no, Dept_no, Dept_name, Job_id, Designation, Salary)

Query:

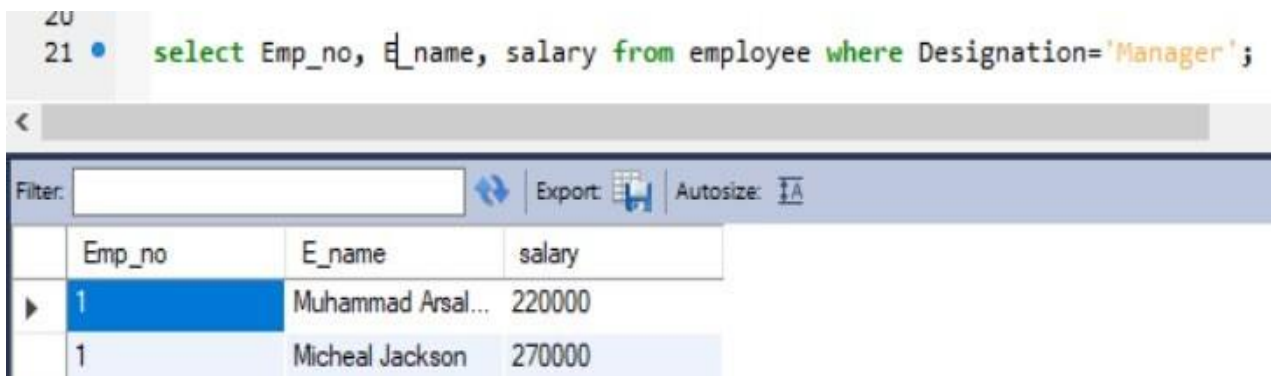


```
1 create database employee;
2 use employee;
3 create table employee(Emp_no int(30), E_name varchar(30), E_address varchar(40), E_ph_no varchar(20), Dept_no varchar(20),
4 Dept_name varchar(30), job_id varchar(20), Designation varchar(20), salary int(6));
5
6 insert into employee values(1, 'Muhammad Arsalan', 'Arsalankhan26666@gmail.com', '0300-3976820', '1', 'ECE', '025',
7 'Student', 20000);
8
9 insert into employee values(1, 'Micheal Jackson', 'Jackson.micheal1@gmail.com', '0300-21158657', '2', 'Cs', '235',
10 'Dean', 270000);
11
12 insert into employee values(1, 'Nauman Arshad', 'ANauman.arshad@outlook.com', '0344-39478564', '3', 'Management', '431',
13 'Lecturer', 110000);
14
15 insert into employee values(1, 'Hamid Ali', 'Hamidkhan@hotmail.com', '0315-1698741', '4', 'Mathematics', '511',
16 'Lab Engineer', 45000);
17
18 insert into employee values(1, 'Muhammad Adil', 'adiladil@hotmail.com', '0314-1290820', '5', 'Sweeping', '740',
19 'Sweeper', 15500);
20
```

Emp_no	E_name	E_address	E_ph_no	Dept_no	Dept_name	job_id	Designation	salary
1	Muhammad Arsalan	Arsalankhan26666@gmail.com	0300-3976820	1	ECE	025	Student	20000
1	Micheal Jackson	Jackson.micheal1@gmail.com	0300-21158657	2	Cs	235	Dean	270000
1	Nauman Arshad	ANauman.arshad@outlook.com	0344-39478564	3	Management	431	Lecturer	110000
1	Hamid Ali	Hamidkhan@hotmail.com	0315-1698741	4	Mathematics	511	Lab Engineer	45000
1	Muhammad Adil	adiladil@hotmail.com	0314-1290820	5	Sweeping	740	Sweeper	15500

Write SQL statements for the following query.

1. List the E_no, E_name, Salary of all employees working for MANAGER.



```
21 select Emp_no, E_name, salary from employee where Designation='Manager';
```

Emp_no	E_name	salary
1	Muhammad Arsal...	220000
1	Micheal Jackson	270000

2. Display all the details of the employee whose salary is more than the Sal of any IT

```
22  
23 • select * from employee where salary>110000;
```

Professor.

Filter:			Export: 	Autosize: 					
	Emp_no	E_name	E_address	E_pho_no	Dept_no	Dept_name	job_id	Designation	salary
	1	Muhammad Arsalan	Arsalankhan26666@gmail.com	0300-3976820	1	ECE	025	Manager	220000
	1	Micheal Jackson	Jackson.micheal1@gmail.com	0300-21158657	2	Cs	235	Manager	270000

3. List the employees who are 'CLERK'.

```
25 • select * from employee where Designation='Clerk';
```

Filter:



Export: 

Autosize: 

	Emp_no	E_name	E_address	E_pho_no	Dept_no	Dept_name	job_id	Designation	salary
▶	1	Muhammad Adil	adiladil@hotmail.com	0314-1290820	5	Sweeping	740	Clerk	15500

Conclusion:

In this lab we implement different type of commands insert, update, delete. We also use in this lab where command for setting condition